

Advanced Programming Image Filters

BarnesHut Group

Code Design

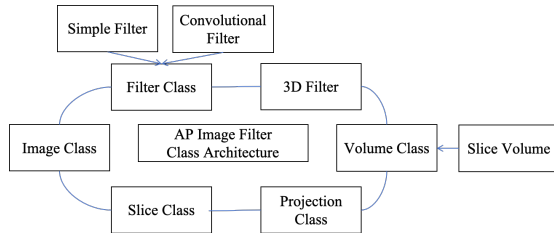


Fig. 1. Implemented class structure overview.

Inheritance, Polymorphism, and Class Interactions

The implemented class structure is shown in Figure 1. Our `Image` and `Volume` classes are concrete value types rather than class hierarchies. Instead of using subclasses like `RGBImage` and `GreyscaleImage`, the channel count is a runtime field, allowing filters to change an image from 3-channel to 1-channel in place without allocating a new image object. Similarly, `Volume` represents depth as a runtime dimension rather than a fixed template parameter, so the same class handles volumes of any size. `Slice` reads from `Volume` and constructs a new `Image` by value, keeping the two classes decoupled.

`SimpleFilter` and `ConvolutionalFilter` inherit from `Filter`, while `Filter3D` is a separate abstract base with `apply(Volume&)`. Concrete filters (e.g. `SobelFilter`, `MedianBlurFilter`) in turn inherit from one of these. This modular design allows new filters to be added by implementing a single `apply` method.

For 2D processing, filters operate on `Image` through the polymorphic `Filter::apply(Image&)` interface. The main function stores filters in a `std::vector<std::unique_ptr<Filter>>` and applies them sequentially, allowing arbitrary filter chains without knowing the concrete type. For 3D processing, `Volume` is loaded from numbered image slices, optionally blurred via `Filter3D`, then reduced to a 2D `Image` through `Projection` or `Slice`. During loading,

`Volume` uses `GreyscaleFilter` to normalise colour slices to single-channel data.

Data Storage and Convolution

The `Image` class stores pixel data in a flat `std::vector<unsigned char>` using row-major, channel-interleaved layout, with index mapping $(y \cdot \text{width} + x) \cdot \text{channels} + c$. In contrast to something like nested arrays, this structure for image and volume data leverages the efficiency of contiguous memory.

Both the `Image` and `Volume` classes include an access function (`getPixel` or `getVoxel`) that returns the nearest boundary pixel or voxel when a coordinate falls outside the domain. This eliminates the need for explicit padding in our convolutional filters. Despite this convenience, we utilize direct array indexing whenever the kernel is guaranteed to lie fully within the domain, only falling back on the access function for border pixels where the kernel extends beyond the boundary. For an example 512x512 image with kernel size 3, over 99% of pixels utilize the faster direct-access path.

For methods with variable kernel sizes like median blur, we enforce a maximum kernel size of half the smallest image dimension, since larger kernels produce diminishing returns and require excessive boundary handling. Errant kernel behavior such as even, negative, or excessively large kernels (kernel size > 99) is also rejected.

Edge Detection

We implemented edge detection as a generic function accepting arbitrary kernel pairs. It is reused by all four methods (Sobel, Prewitt, Scharr, Roberts Cross). The 2x2 RobertsCross kernels are padded to a 3x3 with zeros to maintain compatibility with the shared function. Magnitude outputs are normalized against the theoretical maximum, which is calculated from the kernel weights—the worst-case gradient assumes all positive-weight pixels are 255 and all negative-weight pixels are 0. This is preferable to clamping, which squashes strong edges to 255 and loses relative differences between moderate and strong edges.

This theoretical maximum approach is also preferable to normalizing by the image’s actual maximum magnitude. Although actual-maximum normalization might provide visually striking results, it makes edge strengths incomparable across different images. Our theoretical normalization can produce fainter outputs on low-contrast images, our histogram equalization filter can be chained afterward to redistribute intensities across the full range.

Blur Filters and Sorting Algorithms

For Gaussian blur, using separable passes reduces per-voxel cost from $O(k^3)$ to $O(3k)$ in 3D and from $O(k^2)$ to $O(2k)$ in 2D, where k is the kernel size. Each pass convolves along a single axis using a 1D kernel derived from the same Gaussian distribution, and the interior/border optimization applies within each pass.

In 2D median blur and median projection, we manually implemented a quick select algorithm, since it finds the median in $O(n)$ average time without fully sorting. While quick select is sufficient for 2D, we used a histogram-based approach for our 3D implementation. Since voxel values are integers in $[0, 255]$, we build a 256-bin histogram of the kernel neighborhood and scan it to find the median position. This makes the median-finding step only $O(256)$ regardless of kernel size, which is advantageous for 3D kernels where the number of neighbors grows cubically. For example, a $7 \times 7 \times 7$ 3D kernel has 343 neighbors, making quickselect’s $O(n)$ average less competitive. For 2D and median projection, where neighbor counts are smaller (e.g. 9 for a 3×3 kernel), the reduced overhead of quickselect makes it viable.

For boxblur, the same optimisation separable passes combined with an integral image (cumulative sum table) achieves $O(1)$ per-pixel cost regardless of kernel size.

Volume Loading, Slicing, and Projection

Since volume data is loaded from directories of numbered image slices with nonstandardized naming conventions, the `Volume` class attempts an assortment of zero-paddings and numbering schemes (e.g. start from 0 vs start from 1). To find the total number of slices without checking every index sequentially, we use an exponential search followed by binary search, which doubles the index until a file is not found, then narrowing the range. This reduces

the detection from $O(n)$ file-existence checks to $O(\log n)$.

Since a slice or projection is always applied following the application of blur, our space and time complexities can be reduced by only applying the filter across the relevant subsection of `Volume`. The subclass `Volume:SliceVolume` rotates the volume to the desired plane with depth always equal to the kernel size. This reduces the space complexity of blur application to $O(whk)$ where k is the kernel size instead of $O(whd)$ where w , h , and d are not necessarily the same after rotation. It reduces the time complexity to a function of $M = whk$ instead of N (total voxels).

This optimization is most effective when the difference between the kernel size and the depth of the rotated volume are very different. It will run approximately $\frac{\text{originalDepth}}{\text{kernelSize}}$ times faster. With slight changes to this class (a rotate function that doesn’t take a kernel size) we’d also be able to easily implement projections on any plane of the volume but we did not implement this.

For orthographic projections, each (x, y) pixel is computed independently by scanning the z -axis. MIP and MinIP track a running maximum/minimum, while mean AIP accumulates a sum. Median AIP uses quickselect rather than the histogram approach used in 3D median blur, since the z -column length (equal to volume depth) is typically small enough that quickselect’s lower overhead outweighs the histogram’s constant-time advantage. For color volumes, each channel is projected independently, so a per-channel MIP naturally produces secondary colors where differently-labeled structures overlap in z .

Color Volumes

As noted above, our `Volume` was extended to support color images. Although CT scans are usually greyscale, in biological contexts like fluorescence microscopy, our MIP can display the locations of all dyed proteins within a volume in a single image. For example, three proteins labelled red, green, and blue at different spatial locations produce a colour MIP with distinct regions for each, and yellow, cyan, or white where they overlap.

Performance Analysis

Experimental Setup

We evaluate 2D filters with two benchmarks: runtime vs. image size and runtime vs. kernel size. Image size scales up to 4096×4096 . For kernel analysis, we sweep odd kernel sizes from 3 to 21 on a fixed 1024×1024 image. For 3D filters, we test volumes up to 96^3 and kernel sizes from 3 to 11.

2D Filtering Performance

Simple Filters: Runtime vs. Image Size

Figure 2 reports all five simple filters. Their slopes are roughly 0.8–0.9, close to $O(N)$ which matches the theoretical linear scaling between runtime and pixel count.

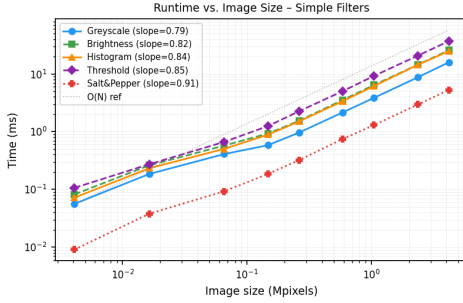


Fig. 2. 2D simple filters: runtime vs. image size.

Convolutional Filters: Runtime vs. Image Size

Figure 3(a) summarizes convolutional performance at fixed kernel size $k = 7$. All convolutional filters have slope near 1.0, following $O(N)$ scaling. *MedianBlur* is 2–3 orders of magnitude slower than the others, because it is not a linear convolution—each pixel requires median selection over a k^2 window, with about $O(k^2)$ work. This is expected since linear convolutions (Sobel, Gaussian, etc.) perform $O(k)$ or $O(k^2)$ operations per pixel, while median selection requires collecting and partitioning the full neighborhood without separability.

Convolutional Filters: Runtime vs. Kernel Size

Figure 3(b) shows runtime against kernel size at fixed 1024×1024 resolution. Only variable-kernel filters are meaningful here. *BoxBlur* has slope ≈ 0.01 (almost $O(1)$), since the sliding-window update uses only constant-time add/subtract per pixel. This confirms the integral image approach achieves kernel-size-independent computation per pixel. *GaussianBlur* has slope ≈ 0.86 (close to

$O(k)$), because separable convolution runs one horizontal and one vertical 1D pass. *MedianBlur* has slope ≈ 2.07 (close to $O(k^2)$), because each pixel processes a full k^2 neighborhood and cannot use separable optimization.

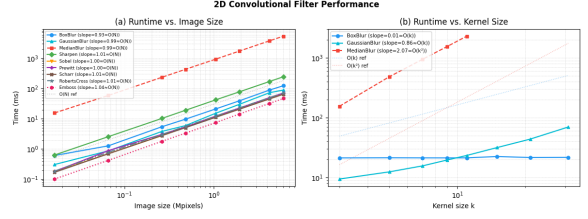


Fig. 3. 2D convolutional filters: (a) runtime vs. image size at fixed $k = 7$; (b) runtime vs. kernel size at fixed 1024×1024 input.

Notably, for *GaussianBlurFilter* uses separable 1D horizontal and vertical passes, reads the packed pixel buffer via raw pointers, keeps the horizontal result in a floating-point buffer between passes, and writes the final unsigned char buffer once.

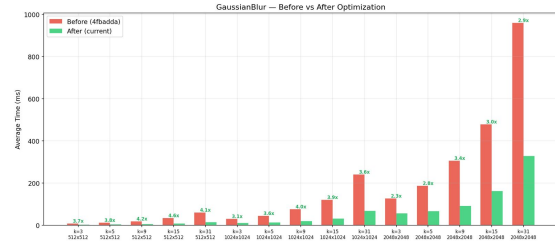


Fig. 4. 2D Gaussian benchmark results.

3D Filter Performance

3D timings were collected with the same C++ harness style as the rest of the suite: builds use C++20 with optimisation ($-O3$). Each run uses a cubic `Volume` filled with deterministic pseudo-random voxels, clones the volume, then applies the 3D filter on the copy so repeated trials see identical input. We report mean runtime over several repetitions (fewer repeats for larger volumes to keep total time manageable) while sweeping volume edge length and odd kernel sizes, and we compare baseline versus optimised implementations on the same configurations.

Gaussian3D and Median3D: Runtime vs. Kernel Size

In the following benchmarks, *baseline* refers to the initial implementation using boundary-checked access (`getVoxel`) for all voxels and

`nth_element` for median selection, while *optimized* refers to the version using direct array indexing for interior voxels and histogram-based median finding.

Figure 5 shows how 3D filter runtime scales with kernel size. For *Gaussian3D* at 64^3 , baseline runtime scales from 1.4 ms at $k = 3$ to 4.5 ms at $k = 11$, while the optimized runtime plateaus near 2.6 ms. For *Median3D*, baseline runtime degrades much more sharply—from 67 ms to 630 ms for $k = 3$ to $k = 7$ —whereas the optimized runtime remains near 40 ms. These results show that the optimization effectively mitigates median filtering’s kernel-growth cost and provides the greatest benefits for larger kernels.

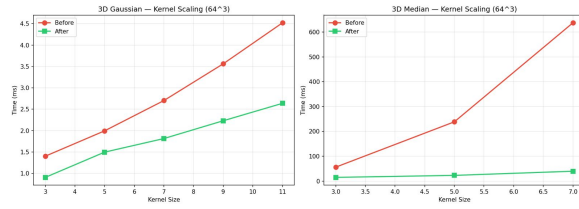


Fig. 5. 3D filter runtime vs. kernel size for Gaussian and median filtering.

Optimization Speedup across Configurations

Figure 6 compares optimized versus baseline performance across all tested configurations. *Median3D* achieves the largest speedups, reaching $16.1\times$ for $k = 7$ at 64^3 resolution, while *Gaussian3D* gains range from $1.3\times$ to $2.1\times$. This gap reflects each algorithm’s computational characteristics: *Median3D*’s non-linear order-statistics operations benefit more from optimization than *Gaussian3D*’s separable convolution path. The consistent speedup pattern across varied kernel sizes and input dimensions indicates that the optimization strategy generalizes well.

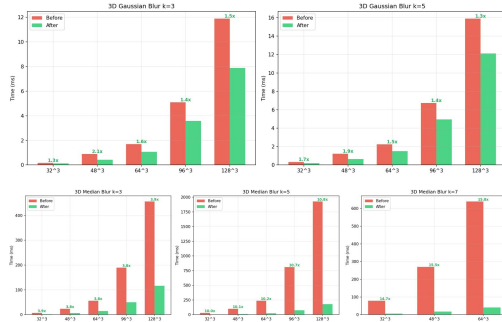


Fig. 6. 3D benchmark comparison with Gaussian (top) and median (bottom).

SliceVolume vs. Full-Volume Processing

In Figure 7, we can see that slice-based processing delivers larger efficiency gains over full-volume computation as problem size increases. For $96 \times 96 \times 96$ Gaussian filtering at $k = 9$, optimization achieves a $10.6\times$ speedup, whereas smaller kernels ($k = 3$) yield only about $0.7\times$ – $5.0\times$ speedups due to the *SliceVolume* implementation. It’s worth noting that for Gaussian $3 \times 16 \times 16$ the slice implementation was actually slightly slower. This is because of the overhead to build a new volume when the kernel size is so similar to the image dimensions. As the volume gets bigger, this overhead becomes less noticeable because the function takes more of the time.

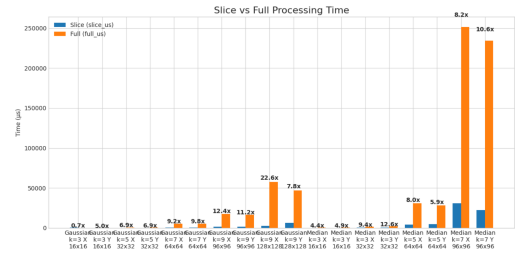


Fig. 7. SliceVolume vs. full-volume processing performance across volume sizes and kernel sizes.

Task Allocation

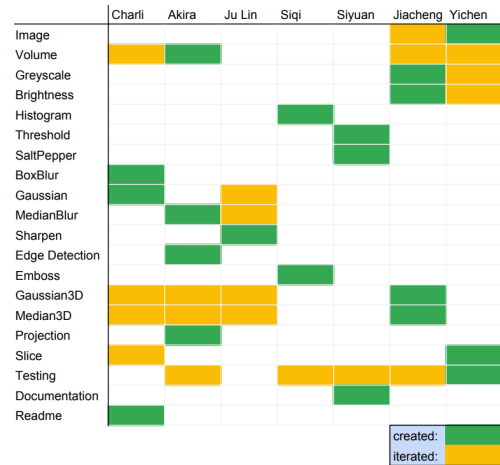


Fig. 8. Task allocation across team members.

References

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. L. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [3] P. Rodríguez-Sevilla et al., “Multichannel fluorescence microscopy: advantages of going beyond a single emission,” *Advanced NanoBiomed Research*, 2022.